

Integración de Servicios Cloud GCP

De los servicios y puertos al control de usuarios y permisos

Semana 05 - Sesión 09

Rel Guzman



**Universidad
Tecnológica
del Perú**

- ▶ ¿Qué etapas recorre un servicio desde que se instala hasta que se da de baja?
- ▶ ¿Cómo se relacionan un servicio que escucha, su puerto y una regla del firewall?
- ▶ ¿Qué registro DNS usarías para apuntar un dominio a la IP pública de una VM?

Logro de la sesión

Al finalizar, podrás reconocer qué hicieron `nginx`, MySQL y el firewall, justificar los accesos de red, diferenciar usuarios normales y privilegiados, aplicar permisos seguros y crear una nueva cuenta de usuario en Ubuntu.

- ▶ En un servidor compartido, ¿todas las cuentas deberían poder hacer lo mismo?
- ▶ ¿Qué diferencia imaginas entre un usuario normal, `root` y una cuenta usada por un servicio?
- ▶ ¿Cómo decidirías quién puede leer, modificar o ejecutar un archivo?

¿Para qué te servirá lo aprendido?

Una aplicación en producción reúne servicios, puertos y cuentas con distintos privilegios. Reconocer cada pieza permite diagnosticar si el problema está en el servicio, en la red o en el acceso asignado a una persona o proceso.

La misma idea se repite en todo el servidor: permitir únicamente lo necesario. Se aplica al firewall, a los roles y a los permisos de scripts, configuraciones y secretos.

De los servicios al control de acceso

Lo que observamos en la Sesión 08

Escuchó en el puerto público 80 y respondió al navegador con el sitio web. El cliente necesitó la IP de la VM y el puerto; `nginx` se encargó de procesar la solicitud HTTP.

Consulta la versión instalada

```
$ nginx -v
```

El número exacto depende de la versión de Ubuntu y de su repositorio.

Versión y año

0.1.0 - 2004

Primera versión pública, publicada el 4 de octubre.

1.0 - 2011

Primera versión estable de la serie 1.x.

La consecuencia

En la práctica final también actuará como **proxy inverso**: el navegador hablará con `nginx` y el puerto 3000 de Python podrá permanecer cerrado al exterior.

Servidor y cliente cumplen funciones distintas

El servidor `mysqld` escucha normalmente en el puerto 3306, procesa instrucciones SQL y conserva los cambios en disco. El programa `mysql` es el cliente que se conecta y envía las consultas.

Consulta las versiones instaladas

```
$ mysqld --version  
$ mysql --version
```

La primera corresponde al servidor; la segunda, al cliente.

Versiones y años clave

Primera versión - 1995

Se publicó en mayo de 1995.

8.0 - 2018

Introdujo una generación moderna del motor.

8.4 LTS - 2024

Primera versión de soporte prolongado.

No debe quedar público

La aplicación necesita conectarse a MySQL; cualquier equipo de Internet, no. El acceso debe limitarse por red y por usuario de la base de datos.

Lo que comprobamos en la Sesión 08

El servicio podía seguir activo aunque una conexión no llegara. El firewall revisaba origen, puerto, protocolo y orden de las reglas antes de permitir o bloquear el tráfico.

¿Qué componente hace cada trabajo

Netfilter forma parte del kernel de Linux y aplica el filtrado. **UFW** es la herramienta sencilla con la que escribimos y consultamos esas reglas.

Instalar no significa activar

Instalar UFW agrega la herramienta. `sudo ufw enable` activa el firewall y hace que sus reglas se carguen al iniciar.

¿Se instalaba o ya estaba disponible?

```
$ command -v ufw
$ ufw --version
$ sudo ufw status
```

Netfilter forma parte del kernel y no se instala aparte. Ubuntu suele incluir UFW, pero se comprueba: si el comando no existe, entonces se instala.

```
$ sudo apt install ufw
```

Una misma IP puede atender muchos servicios a la vez

Por cada dirección IP, TCP y UDP numeran sus puertos del 0 al 65535; el 0 se reserva y los servicios usan del 1 al 65535. El puerto identifica a qué proceso debe entregarse cada conexión.

La misma IP, destinos distintos

22/tcp: SSH administra la VM.

80/tcp: nginx recibe HTTP.

3306/tcp: MySQL recibe consultas.

3000/tcp: la aplicación Python escucha localmente.

El puerto no abre nada por sí solo

Para que una conexión llegue deben coincidir tres condiciones: un proceso escucha, la IP es alcanzable y el firewall autoriza ese tráfico.

No se abren los 65 535

Solo se exponen los puertos indispensables. Cada puerto innecesario aumenta la superficie de ataque.

Cada puerto abierto es superficie de ataque

Su trabajo no es cerrarlo todo, sino justificar cada acceso: qué servicio escucha, quién necesita alcanzarlo, desde dónde y con qué privilegios.

1. Inventariar

```
$ sudo ss -ltnp
```

Identifica qué escucha, en qué dirección y qué programa abrió el puerto.

2. Mirar desde fuera

```
$ nmap -sV VM_IP
```

Observa lo que vería un atacante, incluida la versión del servicio.

3. Justificar el acceso

Decide si debe ser accesible desde Internet, desde la red interna o solo desde la propia máquina.

4. Reducir y vigilar

Cierra lo innecesario, restringe por origen, mantiene los servicios actualizados y revisa los registros.

¿Qué roles recuerdan y qué acceso necesita cada uno?

Relaciona cada rol con una responsabilidad

Desarrollador, SysAdmin, DevOps/SRE, cloud engineer, arquitecto cloud, DBA y analista de seguridad. ¿Cuál escribe la aplicación, cuál opera el servidor y cuál decide o audita los accesos?

Pregunta para discutir

¿Un desarrollador debe tener el mismo nivel de acceso que un administrador del sistema (SysAdmin)?

Piensa en producción

¿Quién debería instalar paquetes, crear usuarios, cambiar el firewall o reiniciar cualquier servicio?

Piensa en la aplicación

¿Qué archivos, registros y comandos necesita realmente un desarrollador para desplegar y diagnosticar su código?

Criterio para la siguiente sección

No necesitan el mismo acceso. Cada persona y servicio debe recibir solo los permisos indispensables para cumplir su función: es el **principio de mínimo privilegio**.

Gestión de permisos

El control de acceso continúa dentro del servidor

Idea central

Ubuntu protege cada archivo indicando qué acciones puede realizar el **propietario**, el **grupo** y el **resto de usuarios**. Un servidor lo usan varias personas y varios servicios: los permisos evitan que un programa cualquiera lea las credenciales de otro.

¿Quién?

u: propietario.

g: grupo.

o: otros usuarios.

¿Qué puede hacer?

r: leer.

w: modificar.

x: ejecutar o atravesar un directorio.

Muestra las cuentas de usuario del sistema

```
$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
rgap:x:1000:1000:Rel Guzman:/home/rgap:/bin/bash
```

Cada línea representa una cuenta. El resultado incluye personas, componentes del sistema y servicios; los nombres y números exactos cambian entre máquinas.

¿Por qué todos pueden leerlo?

El sistema y muchos programas necesitan relacionar los UID y GID con nombres de cuentas. Este archivo contiene información de identificación, no las contraseñas de los usuarios.

Datos que puedes reconocer

usuario: nombre de la cuenta.

UID: identificador del usuario.

GID: grupo principal.

descripción: nombre o comentario.

home: carpeta inicial.

shell: programa que se inicia al entrar.

Cuentas usadas por personas

Root user: `root`, UID 0. Tiene control total; se reserva para administración excepcional y recuperación.

Regular user: representa a una persona. Normalmente tiene UID desde 1000, carpeta personal y shell interactiva.

Administrator user: usuario regular autorizado a usar `sudo`. No es otro tipo de UID; eleva privilegios cuando los necesita.

Cuentas usadas por el sistema

System user: ejecuta tareas internas. Suele tener UID bajo y una shell que impide entrar.

Service account: cuenta dedicada a un servicio, como `www-data` o `mysql`. Limita lo que ese proceso puede controlar.

Las de mayor impacto

`root` y los administradores pueden cambiar todo. Las cuentas de servicios expuestos también son críticas y deben tener el mínimo privilegio.

¿Se puede iniciar sesión con cualquiera?

No. `/bin/bash` solo indica que una sesión interactiva es posible; también se necesitan credenciales, una cuenta desbloqueada y una política que lo permita. `/usr/sbin/nologin` o `/bin/false` la rechazan. Incluso `root` puede tener prohibido el acceso por SSH.

Los archivos del proyecto (carpeta Sesion08/proyecto)

```
rgap@ubuntu:~/proyecto$ ls -la
-rw-rw-r-- 1 rgap rgap  79 Sep 1 13:51 .env
-rw-rw-r-- 1 rgap rgap 2845 Sep 1 13:50 servidor_http.py
```

Con `ls -a` también aparecen los archivos ocultos, los que empiezan con punto.

-	rw-	rw-	r--
Archivo regular. Una d indicaría directorio.	El propietario lee y modifica; todavía no ejecuta.	El grupo rgap lee y modifica.	Los demás solo leen. Aquí está el problema del .env.

Caso 1: un .py necesita intérprete y permiso x

Primera línea del script

```
#!/usr/bin/env python3
from http.server import \
    BaseHTTPRequestHandler, HTTPServer
...
```

Le indica a Ubuntu qué programa debe interpretarlo.

Por qué python3 archivo.py sí funciona

Ahí ejecutas Python, y Python solo lee el archivo.
./archivo.py exige que el archivo mismo sea ejecutable.

Antes y después

```
$ ./servidor_http.py
bash: Permiso denegado
$ chmod u+x servidor_http.py
$ ./servidor_http.py
Escuchando en
http://127.0.0.1:8080
```

Caso 2: el .env contiene secretos

Qué contiene

```
DB_USER=app_web
DB_PASSWORD=cambia-esta-clave
API_KEY=clave-de-ejemplo-no-real
```

Credenciales que dan acceso real a la base de datos y a servicios de pago.

Ciérralo al propietario

```
$ chmod 600 .env
$ ls -l .env
-rw----- 1 rgap rgap 223 .env
```

600: el propietario lee y escribe; nadie más entra.

El permiso no reemplaza al .gitignore

chmod protege el archivo en esta máquina. Para que no llegue al repositorio, además debe estar listado en .gitignore.

¿Qué es chmod?

Viene de *change mode*. Modifica quién puede leer, escribir o ejecutar un archivo o directorio; no cambia su contenido ni su propietario.

u+x: forma simbólica

```
$ chmod u+x app.py
```

u: usuario propietario.

+: agrega un permiso.

x: permite ejecutar.

El propietario podrá ejecutar `app.py`; sus otros permisos no cambian.

Dos formas de expresar el cambio

u+x agrega un permiso concreto; 600 establece los permisos completos de propietario, grupo y otros.

600: forma numérica

```
$ chmod 600 .env
```

6=4+2: el propietario lee y escribe.

Primer 0: el grupo no tiene permisos.

Segundo 0: los demás tampoco.

Simbólica: dices qué cambiar

```
$ chmod u+x app.py  
$ chmod o-r .env
```

La primera agrega ejecución al propietario. La segunda quita la lectura a los demás.

Los tres que verás casi siempre

```
chmod 600 .env rw- --- --- solo el propietario  
chmod 644 index.html rw- r-- r-- todos leen, solo tú editas  
chmod 777 archivo rwx rwx rwx todos hacen todo: evítalo
```

777 no arregla la causa de un error de permisos; solo amplía quién puede provocarlo.

Numérica: $r=4$, $w=2$, $x=1$

```
4+2+1 = 7 rwx  
4+2 = 6 rw-  
4 = 4 r--
```

Un dígito por cada grupo: propietario, grupo y otros.

1. Crea sitio.py y Pruébalo

```
from http.server import \
    BaseHTTPRequestHandler, HTTPServer
class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header('Content-Type',
            'text/html; charset=utf-8')
        self.end_headers()
        self.wfile.write(HTML.encode())
HTTPServer(("127.0.0.1", 3000),
    Handler).serve_forever()
```

El HTML va en la constante HTML, arriba del todo.

2. Ponlo detrás de nginx

```
$ sudo nano \
    /etc/nginx/sites-available/default
location / {
    proxy_pass http://127.0.0.1:3000;
}
$ sudo nginx -t
$ sudo systemctl reload nginx
```

3. Corrige los permisos

```
$ chmod u+x iniciar.sh
$ chmod 600 .env
```

Comprobación

Desde el host, `http://VM_IP` muestra la página con el 3000 cerrado en `ufw`: quien atiende afuera es `nginx`. Explica por qué el 3000 no necesita abrirse.

¿Qué es un reverse proxy?

Recibe la solicitud del cliente, la reenvía a un servidor interno y devuelve su respuesta. El navegador solo ve a nginx; el puerto de la aplicación permanece interno.

El recorrido de la solicitud

```
Navegador -> nginx :80/:443 -> sitio.py 127.0.0.1:3000  
Navegador <- nginx <----- respuesta generada por Python
```

nginx: web server público

Atiende HTTP y HTTPS, entrega archivos estáticos y funciona como punto de entrada. También puede gestionar TLS, límites, registros y balanceo de carga.

sitio.py: application server

Sí: en esta práctica ejecuta la lógica Python y genera la respuesta dinámica en 127.0.0.1:3000.

Precisión: usa `http.server` para aprender el flujo; no es un servidor de producción. Una aplicación real usa un servidor especializado para su framework.

Usuarios del sistema

Comprueba tu identidad

```
rgap@ubuntu:~$ whoami
rgap
rgap@ubuntu:~$ id
uid=1000(rgap) gid=1000(rgap)
groups=1000(rgap),27(sudo)
```

rgap es el **Regular user** o usuario normal con el que se inició sesión. Su UID no es 0.

Lo que ocurrió durante la práctica

Los comandos sin sudo se ejecutaron como rgap. En `sudo nginx -t`, solo ese comando recibió privilegios de administrador.

Root user o superusuario

root tiene UID 0 y control total sobre el sistema. Puede modificar archivos, usuarios, servicios y reglas de red.

Administrator user

Sigue siendo un usuario regular, pero pertenece al grupo `sudo`. Puede elevar un comando cuando una tarea administrativa lo requiere.

Abrir y cerrar una terminal como root

```
rgap@ubuntu:~$ sudo -i
[sudo] password for rgap:
root@ubuntu:~# whoami
root
root@ubuntu:~# pwd
/root
root@ubuntu:~# exit
logout
rgap@ubuntu:~$
```

Qué cambió en la terminal

Se pidió la contraseña de rgap. El prompt cambió de \$ a #, whoami mostró root y la carpeta personal pasó a /root. exit devolvió la sesión a rgap.

¿Es recomendado?

Para una sola tarea, se recomienda sudo comando. Usa sudo -i solo cuando necesites realizar varias tareas administrativas y sal con exit al terminar. No es recomendable trabajar siempre como root ni habilitar su inicio de sesión directo.

1. Crea un usuario regular

```
rgap@ubuntu:~$ sudo adduser juandev
Adding user `juandev' ...
Adding new group `juandev' ...
Creating home directory `/home/juandev' ...
New password:
Retype new password:
Is the information correct? [Y/n] Y
rgap@ubuntu:~$ id juandev
uid=1001(juandev) gid=1001(juandev)
groups=1001(juandev)
```

2. Inicia sesión con esa cuenta

```
rgap@ubuntu:~$ su - juandev
Password:
juandev@ubuntu:~$ whoami
juandev
juandev@ubuntu:~$ pwd
/home/juandev
juandev@ubuntu:~$ exit
logout
rgap@ubuntu:~$
```

Explica lo que ocurrió

adduser creó la cuenta, su grupo principal y /home/juandev; también pidió una contraseña. su - juandev inició una sesión con esa identidad y cargó su entorno. juandev no es administrador: no pertenece a sudo salvo que se agregue explícitamente.

El equipo comparte el servidor, no la misma cuenta

Un equipo con responsabilidades diferentes

```
admin@empresa.com -> SysAdmin / Administrador  
juan@empresa.com -> Developer  
maria@empresa.com -> Developer  
pedro@empresa.com -> DevOps
```

¿Por qué podrían entrar al servidor?

Admin: instala paquetes, crea usuarios y mantiene el sistema.

Juan y María: despliegan la aplicación y consultan sus registros.

Pedro: automatiza despliegues y atiende incidentes operativos.

¿Qué se hace en ese caso?

Se crea una cuenta Linux individual, se autentica con SSH, se asignan grupos y permisos según el rol, y se concede `sudo` solo para las tareas que realmente lo requieren. Los accesos se registran y se revocan cuando dejan de ser necesarios.

Acceso necesario no significa acceso total

Nunca se comparte la cuenta `root` ni una contraseña entre el equipo. Si un despliegue puede hacerse mediante una automatización, el desarrollador quizá no necesite acceso SSH directo.

SSH abre una terminal segura en el servidor remoto

SSH: Secure Shell

Crea una conexión cifrada entre tu computadora y una cuenta del servidor. Por defecto usa el puerto 22/tcp; lo que escribes se ejecuta en la máquina remota.

juan@laptop → **SSH cifrado** → juan@VM

La forma básica

```
$ ssh juan@203.0.113.10
```

ssh: programa cliente.

juan: cuenta en la VM.

203.0.113.10: IP del servidor.

Si no se indica otro, usa el puerto 22.

Primera conexión y salida

```
juan@laptop:~$ ssh juan@203.0.113.10
Continue connecting (yes/no)? yes
juan@203.0.113.10's password:
juan@vm:~$ whoami
juan
juan@vm:~$ exit
Connection to 203.0.113.10 closed.
```

Antes de responder `yes`, verifica la huella. Al entrar, el prompt cambia al de la VM; `exit` cierra la conexión.



gracias



Universidad
Tecnológica
del Perú

- ▶ NGINX Community Blog: *Celebrating 20 Years of NGINX y Announcing NGINX Unit 1.0*, blog.nginx.org.
- ▶ Documentación de NGINX: *Reverse Proxy* y guía del servidor web, docs.nginx.com/nginx/admin-guide/web-server.
- ▶ Documentación de Python: módulo `http.server`, docs.python.org/3/library/http.server.html.
- ▶ Oracle / MySQL: *30 Years MySQL Timeline*, mysql.com/news-and-events/events/mysql-30th.
- ▶ Documentación de Ubuntu Server: *Firewall*, documentation.ubuntu.com/server/how-to/security/firewalls.
- ▶ IANA: *Service Name and Transport Protocol Port Number Registry*, iana.org/assignments/service-names-port-numbers.
- ▶ Páginas de manual de `nginx`, `mysqld`, `mysql`, `ufw`, `ss`, `nmap`, `passwd`, `nologin`, `sudo`, `su`, `adduser`, `ssh`, `ssh-keygen`, `id`, `whoami`, `ls` y `chmod`.
- ▶ Material base proporcionado: Sesión 08; identidad visual UTP y estructura docente reutilizadas sin cambios.